

مقایسه رویکردهای سنتی و مدرن

مقایسه مدل فرآیند توسعه

مدل آبشاری دارای مراحل مشخصی مانند تحلیل، طراحی، پیاده‌سازی، آزمون و استقرار است و هر مرحله پس از تکمیل مرحله قبل آغاز می‌شود. مزیت اصلی این مدل، مستندسازی دقیق و کنترل مناسب فرآیند توسعه است اما پس از بررسی ویژگی‌های سامانه آموزشی، مشخص شد که نیازمندی‌های چنین سیستمی در طول زمان ثابت باقی نمی‌ماند. امکانات جدید، تغییر قوانین آموزشی، نیازهای کاربران، بهبود رابط کاربری و توسعه سرویس‌ها به صورت مستمر اتفاق می‌افتد. در نتیجه استفاده از مدل آبشاری باعث می‌شود اعمال تغییرات هزینه و زمان زیادی داشته باشد.

به همین دلیل، رویکرد چابک مبتنی بر Scrum برای این پروژه مناسب‌تر تشخیص داده شد. در این روش توسعه سیستم به چندین Sprint کوتاه تقسیم می‌شود و در پایان هر Sprint نسخه قابل استفاده‌ای از سامانه تولید خواهد شد. از طرف دیگر، برای استقرار سریع نسخه‌های جدید و کاهش خطاهای ناشی از انتشار نرم‌افزار، استفاده از DevOps نیز در کنار Agile پیشنهاد شد. DevOps باعث یکپارچه شدن تیم توسعه و عملیات شده و فرآیند Build ، Test و Deploy را تا حد زیادی خودکار می‌کند.

بنابراین با توجه به ماهیت پویا و در حال تغییر سامانه آموزشی، گروه ما رویکرد Agile همراه با Scrum و DevOps را مناسب‌ترین گزینه برای توسعه سیستم انتخاب کرد.

مقایسه معماری سیستم

بررسی روش های سنتی:

در معماری یک‌هسته‌ای تمامی بخش‌های سامانه آموزشی از جمله مدیریت کاربران، احراز هویت، مدیریت دوره ها، ثبت نام، آزمون‌ها، تکالیف، نمره‌دهی و سیستم اعلان‌ها، در قالب یک برنامه واحد توسعه و استقرار می‌یابند. اگرچه این معماری برای سامانه‌های آموزشی کوچک با تعداد کاربران محدود می‌تواند گزینه‌ای ساده و کم‌هزینه باشد اما با افزایش تعداد دانشجویان، اساتید و امکانات سامانه، محدودیت‌های آن به تدریج آشکار می‌شود. در چنین ساختاری وابستگی شدید میان ماژول‌ها باعث می‌شود که اعمال تغییر در یکی از بخش‌ها، مانند افزودن قابلیت جدید به سیستم آزمون‌های آنلاین یا تغییر در فرآیند ثبت نام، احتمال بروز اختلال در سایر بخش‌ها را افزایش دهد. علاوه بر این تیم‌های توسعه نمی‌توانند به صورت مستقل روی بخش‌های مختلف سامانه فعالیت کنند و هر تغییر نیازمند ساخت و استقرار مجدد کل سیستم خواهد بود. برای مثال اگر تنها ماژول آزمون‌ها در زمان برگزاری امتحانات پایان‌ترم با افزایش شدید کاربران مواجه شود، در معماری یک‌هسته‌ای امکان افزایش مقیاس همان بخش به صورت مستقل وجود ندارد و کل سامانه باید مقیاس‌پذیر شود که این موضوع هزینه منابع و نگهداری را به طور قابل توجهی افزایش می‌دهد. به همین دلیل معماری یک‌هسته‌ای برای سامانه‌های آموزشی بزرگ و پرتراфик، گزینه‌ای مناسب محسوب نمی‌شود.

اولین مزیتی که برای معماری لایه‌ای شناسایی کردیم سادگی طراحی در توسعه اولیه بود. همچنین به دلیل تفکیک مسئولیت‌ها، فرآیند توسعه برای پروژه‌های کوچک و متوسط نسبتاً ساده‌تر است و بسیاری از ابزارهای توسعه نیز از این معماری پشتیبانی می‌کنند. فرآیند تست و اشکال‌زدایی هم در مراحل ابتدایی توسعه ساده‌تر انجام می‌شود.

چالش‌ها و مشکلات برای سامانه ما:

با وجود این مزایای زمانی که ویژگی‌های سامانه آموزشی را با این معماری تطبیق دادیم محدودیت‌های مهمی مشاهده شد. اولین مشکل وابستگی زیاد لایه‌ها به یکدیگر بود. اگر تغییری در یکی از بخش‌های اصلی سیستم ایجاد شود احتمال تأثیر آن بر سایر لایه‌ها نیز وجود دارد و این موضوع فرآیند توسعه و نگهداری را در بلندمدت دشوار می‌کند.

چالش بعدی مقیاس‌پذیری بود. در معماری لایه‌ای معمولاً کل سامانه به صورت یک واحد اجرا می‌شود. بنابراین اگر تنها یکی از بخش‌ها مانند "برگزاری کلاس آنلاین" با افزایش شدید کاربران مواجه شود امکان افزایش منابع فقط برای همان بخش وجود ندارد و معمولاً باید کل سیستم مقیاس‌دهی شود. این موضوع علاوه بر افزایش هزینه، باعث استفاده غیربهبوده از منابع سخت‌افزاری نیز خواهد شد.

از طرف دیگر توسعه هم‌زمان توسط چند تیم هم در این معماری با محدودیت همراه است چون بیشتر بخش‌های سیستم به یکدیگر وابسته اند و تغییرات ایجاد شده توسط یک تیم ممکن است روی فعالیت سایر تیم‌ها نیز اثر بگذارد و هماهنگی میان اعضای پروژه را پیچیده‌تر کند.

بررسی روش‌های مدرن:

در معماری سرویس‌گرا سیستم به مجموعه‌ای از سرویس‌های مستقل تقسیم می‌شود که هر سرویس مسئول انجام یک وظیفه مشخص است و از طریق رابط‌های استاندارد با سایر سرویس‌ها ارتباط برقرار می‌کند. ساختار سامانه آموزشی ما کاملاً با این نوع معماری سازگار است. قابلیت‌هایی مانند مدیریت کاربران، احراز هویت، کلاس آنلاین و ... هر کدام ماهیت مستقلی دارند و می‌توانند به صورت سرویس‌های جداگانه طراحی شوند. به همین دلیل، معماری سرویس‌گرا امکان جداسازی مناسب این بخش‌ها را فراهم می‌کند.

- | | |
|------------------------|---------------------|
| - سرویس احراز هویت | - سرویس کلاس آنلاین |
| - سرویس آزمون | - سرویس تکالیف |
| - سرویس پیام‌رسان | - سرویس اعلان‌ها |
| - سرویس مدیریت فایل | - سرویس گزارش‌گیری |
| - سرویس مدیریت کاربران | |

هر کدام از این بخش‌ها می‌توانند به صورت مستقل توسعه، تست و استقرار شوند.

یکی از مهم‌ترین مزیت‌هایی که در بررسی‌های گروه به آن رسیدیم **کاهش وابستگی** میان بخش‌های مختلف سیستم بود. هر سرویس بدون ایجاد تغییر در سایر سرویس‌ها قابل توسعه، به‌روزرسانی و استقرار است. این ویژگی باعث می‌شود فرآیند نگهداری سیستم در آینده بسیار ساده‌تر شود و اضافه شدن قابلیت‌های جدید نیز با ریسک کمتری انجام گیرد.

در زمان برگزاری امتحانات آنلاین فشار اصلی روی سرویس آزمون وارد می‌شود در حالی که سرویس‌هایی مانند مدیریت تکالیف یا تقویم آموزشی بار چندانی ندارند. در معماری سرویس‌گرا تنها سرویس آزمون می‌تواند به صورت مستقل **مقیاس‌دهی** شود و نیازی به افزایش منابع کل سامانه نخواهد بود. این موضوع باعث کاهش هزینه و استفاده بهینه‌تر از منابع می‌شود.

همچنین این معماری **امکان توسعه موازی** را نیز فراهم می‌کند. هر تیم می‌تواند روی یک یا چند سرویس مشخص کار کند، بدون اینکه تغییرات آن مستقیماً سایر بخش‌های سیستم را تحت تاثیر قرار دهد. این ویژگی برای پروژه‌هایی که در آینده توسط تیم‌های بزرگ‌تر توسعه پیدا می‌کنند اهمیت بسیار زیادی دارد.

چالش روش مدرن برای سامانه ما و چگونگی رفع آن:

در کنار مزایای معماری سرویس‌گرا چالش‌های آن نیز در طراحی سامانه آموزشی مورد بررسی قرار گرفت. یکی از مهم‌ترین چالش‌ها افزایش پیچیدگی ارتباط میان سرویس‌ها است. در سامانه آموزشی بخش‌هایی مانند مدیریت کاربران، انتخاب واحد، کلاس‌های آنلاین، آزمون، تکالیف و پرداخت شهریه هر کدام به‌صورت یک سرویس مستقل عمل می‌کنند. با افزایش تعداد این سرویس‌ها، مدیریت نحوه ارتباط آن‌ها، کنترل دسترسی، تامین امنیت، کشف سرویس‌ها و مدیریت خطاها به‌مرور دشوارتر می‌شود. برای مثال زمانی که یک دانشجو قصد شرکت در آزمون آنلاین را دارد سرویس آزمون باید اطلاعات هویتی دانشجو را از سرویس کاربران دریافت کند، وضعیت انتخاب واحد را از سرویس آموزشی بررسی کند و در صورت نیاز اطلاعات مربوط به کلاس را نیز از سرویس مربوطه دریافت نماید. اگر این ارتباطات بدون یک ساختار مشخص انجام شوند مدیریت و نگهداری سامانه به‌تدریج پیچیده شده و احتمال بروز خطا افزایش می‌یابد.

برای کاهش این پیچیدگی تصمیم گرفتیم از API Gateway به‌عنوان نقطه ورود تمامی درخواست‌ها استفاده کنیم تا تمام ارتباطات از یک مسیر متمرکز و کنترل‌شده انجام شوند. همچنین برای ایجاد ارتباطی استاندارد و یکپارچه بین سرویس‌ها، استفاده از REST API در نظر گرفته شد. این رویکرد علاوه بر ساده‌تر کردن فرآیند توسعه و نگهداری، امکان اعمال سیاست‌های امنیتی، مدیریت درخواست‌ها و کنترل خطاها را نیز به شکل متمرکز فراهم می‌کند و باعث می‌شود سرویس‌های مختلف سامانه آموزشی با وجود استقلال، هماهنگی مناسبی با یکدیگر داشته باشند.

یکی دیگر از چالش‌های مهم در معماری سرویس‌گرا، مدیریت امنیت در ارتباط میان سرویس‌ها است. از آنجایی که هر بخش از سامانه آموزشی به صورت یک سرویس مستقل فعالیت می‌کند تمامی ارتباطات بین این سرویس‌ها باید به صورت ایمن انجام شوند تا از دسترسی غیرمجاز یا افشای اطلاعات جلوگیری شود. برای مثال زمانی که دانشجو پس از ورود به سامانه قصد مشاهده نمرات یا شرکت در آزمون آنلاین را دارد، درخواست او باید بین سرویس احراز هویت، سرویس کاربران و سرویس آزمون جابه‌جا شود. در صورتی که این ارتباطات به درستی ایمن سازی نشوند امکان سوءاستفاده از اطلاعات کاربران یا دسترسی افراد غیرمجاز به داده‌های آموزشی وجود خواهد داشت. به همین دلیل در معماری پیشنهادی از احراز هویت مبتنی بر JWT برای اعتبارسنجی کاربران، ارتباطات رمزنگاری شده از طریق HTTPS برای حفاظت از داده‌های در حال انتقال و همچنین کنترل دسترسی مبتنی بر نقش (RBAC) استفاده شده است تا هر کاربر تنها به بخش‌هایی از سامانه که متناسب با نقش او است مانند دانشجو، استاد یا مدیر، دسترسی داشته باشد.

موضوع دیگری که در طراحی سامانه مورد توجه قرار گرفت، مانیتورینگ و مدیریت خطاها بود. در معماری سرویس‌گرا، برخلاف معماری لایه‌ای، یک درخواست ممکن است برای تکمیل شدن از چندین سرویس مختلف عبور کند؛ به همین دلیل شناسایی محل دقیق بروز خطا پیچیده‌تر خواهد بود. مثلاً اگر دانشجو هنگام ثبت نام در یک درس با خطا مواجه شود این مشکل ممکن است از سرویس انتخاب واحد، سرویس کاربران، سرویس پرداخت یا حتی پایگاه داده یکی از این سرویس‌ها ناشی شده باشد. برای مدیریت این وضعیت استفاده از سیستم‌های Logging جهت ثبت رویدادها و همچنین Distributed Tracing برای ردیابی مسیر حرکت درخواست‌ها بین سرویس‌های مختلف در معماری پیشنهادی در نظر گرفته شد. این راهکارها موجب می‌شوند محل بروز خطا با سرعت بیشتری شناسایی شده و فرآیند عیب‌یابی و نگهداری سامانه آموزشی با دقت و کارایی بالاتری انجام شود.

انتخاب نهایی:

اما در نهایت و با همه ی این چالش‌ها به خاطر گستردگی قابلیت‌ها، تعداد زیاد کاربران، نیاز به توسعه مداوم، مقیاس‌پذیری، نگهداری آسان و امکان توسعه مستقل بخش‌های مختلف باعث شد معماری سرویس‌گرا را به عنوان معماری اصلی سیستم انتخاب کنیم. این معماری علاوه بر پاسخگویی به نیازهای فعلی سامانه، امکان توسعه و پشتیبانی آن را در سال‌های آینده نیز به شکل مؤثرتری فراهم خواهد کرد.

تحلیل و بحث cloud:

یکی دیگر از موضوعاتی مورد بحث بود و بررسی شد محل استقرار سیستم بود. اگر سامانه بر روی یک سرور ثابت اجرا شود در زمان‌هایی مانند امتحانات پایان ترم یا برگزاری کلاس‌های همزمان، احتمال افزایش شدید بار سیستم وجود خواهد داشت. برای رفع این مشکل، استفاده از زیرساخت ابری پیشنهاد شد. پس از بررسی معماری Cloud-Native به این نتیجه رسیدیم که استفاده از معماری آن صرفاً یک انتخاب مبتنی بر فناوری‌های جدید نیست بلکه پاسخی به نیازهای واقعی سیستم است.

مزایای cloud برای سامانه ما:

در بررسی سامانه آموزشی، مشاهده کردیم که الگوی استفاده کاربران در طول ترم یکنواخت نیست. برای مثال در روزهای عادی تنها تعداد محدودی از دانشجویان وارد سامانه می‌شوند اما در زمان برگزاری امتحانات پایان‌ترم یا کلاس‌های آنلاین، تعداد کاربران همزمان به‌طور ناگهانی افزایش پیدا می‌کند. اگر سامانه روی یک زیرساخت سنتی اجرا شود برای پاسخگویی به این بار ناگهانی باید منابع کل سیستم افزایش یابد در حالی که بیشتر بخش‌ها مانند مدیریت تکالیف یا پیام‌رسان در همان زمان بار زیادی ندارند. به همین دلیل استفاده از Cloud Native این امکان را فراهم می‌کند که تنها سرویس آزمون یا کلاس آنلاین به‌صورت مستقل مقیاس دهی شود و سایر سرویس‌ها بدون تغییر به فعالیت خود ادامه دهند در نتیجه منابع سخت‌افزاری به شکل بهینه مصرف شده و هزینه‌های عملیاتی نیز کاهش می‌یابد.

یکی از نیازهای اصلی سامانه آموزشی دسترس‌پذیری در زمان برگزاری کلاس‌ها و آزمون‌های آنلاین است. اگر در زمان امتحان یکی از نمونه‌های سرویس آزمون دچار مشکل شود توقف کامل سامانه می‌تواند باعث اختلال در فرآیند برگزاری آزمون شود. در معماری Cloud Native هر سرویس در چند نمونه اجرا می‌شود. بنابراین در صورت از کار افتادن یکی از نمونه‌ها، درخواست کاربران توسط نمونه‌های دیگر پاسخ داده می‌شود و آزمون بدون وقفه ادامه پیدا می‌کند. این ویژگی ریسک توقف سامانه در زمان‌های حساس را به میزان قابل توجهی کاهش می‌دهد.

در این سامانه ممکن است در یک ترم تنها قوانین مربوط به بخش آزمون تغییر کند یا در ترم بعد قابلیت جدیدی به کلاس آنلاین اضافه شود. در معماری Cloud Native این تغییرات تنها روی همان سرویس اعمال می‌شود و نیازی به استقرار مجدد کل سامانه وجود ندارد. بنابراین تیم توسعه می‌تواند قابلیت‌های جدید را با سرعت بیشتر و ریسک کمتر منتشر کند بدون اینکه عملکرد سایر بخش‌ها مانند مدیریت کاربران یا تکالیف تحت تاثیر قرار گیرد.

بررسی نحوه استفاده از سامانه نشان می‌دهد که بار کاری آن در طول سال ثابت نیست. در ابتدای ترم یا زمان انتخاب واحد و همچنین هفته امتحانات، مصرف منابع به شکل محسوسی افزایش پیدا می‌کند اما در سایر زمان‌ها بخش زیادی از این منابع بلااستفاده خواهند ماند. در زیرساخت Cloud منابع متناسب با میزان استفاده افزایش یا کاهش پیدا می‌کنند. بنابراین دانشگاه تنها هزینه منابعی را پرداخت می‌کند که واقعا مصرف شده‌اند و نیازی به تهیه سخت‌افزار با ظرفیت حداکثری نخواهد بود.

همچنین Cloud-Native به دلیل استفاده از کانتینرها، ارکستریشن و خودکارسازی فرآیندهای استقرار، امکان استانداردسازی محیط‌های اجرایی را فراهم می‌کند. این موضوع باعث می‌شود اختلاف بین محیط توسعه، آزمایش و تولید به حداقل برسد و بسیاری از خطاهایی که ناشی از تفاوت محیط‌ها هستن حذف بشه.

از دیدگاه آینده‌نگری هم معماری Cloud-Native برای توسعه تدریجی سامانه بستری مناسب فراهم می‌کند. در صورتی که در آینده قابلیت‌های جدیدی به سیستم اضافه شود یا نیازهای کسب‌وکار تغییر کند، می‌توان سرویس‌های جدید را بدون ایجاد تغییرات اساسی در سایر بخش‌های سیستم توسعه داد. این ویژگی موجب افزایش قابلیت توسعه و کاهش هزینه تغییرات در بلندمدت می‌شود.

در مجموع انتخاب معماری Cloud-Native نتیجه بررسی دقیق نیازهای عملکردی و غیرعملکردی سیستم است. قابلیت مقیاس‌پذیری، دسترس‌پذیری بالا، توسعه مستقل سرویس‌ها، کاهش هزینه‌های عملیاتی، مدیریت بهینه منابع و آمادگی برای توسعه‌های آینده، همگی عواملی هستند که این معماری را نسبت به معماری‌های سنتی به گزینه‌ای مناسب‌تر برای این پروژه تبدیل می‌کنند و در طراحی نهایی سامانه معماری Cloud-Native به عنوان بستر اصلی استقرار و اجرای سرویس‌ها انتخاب شد.